ORACLE Certified Professional

IBM DIGITAL - NATION Python

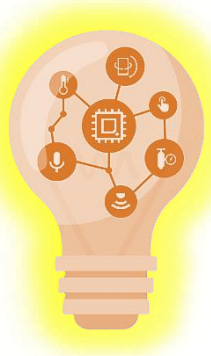
IBM DIGITAL - NATION MACHINE LEARNING V2

IBM DIGITAL - NATION DATA SCIENCE FUNDAMENTALS

Java Oracle Certified Professional Java Programmer

Microsoft Certified Professional

AwithSI سُہیل عِمران



Dr.SOHAIL IMRAN

OOP adv.

sohailimran2@yahoo.com

Ref: Web, Slides UI through NotebookLM

Case Study 1: Strong Relationship

Concept: Composition represents a "part-of" relationship where the component's lifecycle is strictly tied to the container, and the container's core functionality depends on it.

The Scenario: In this model, a **Vehicle has an Engine**. The engine is considered an **integral component** because the vehicle's primary function—to **move()**—is impossible without a functional engine.

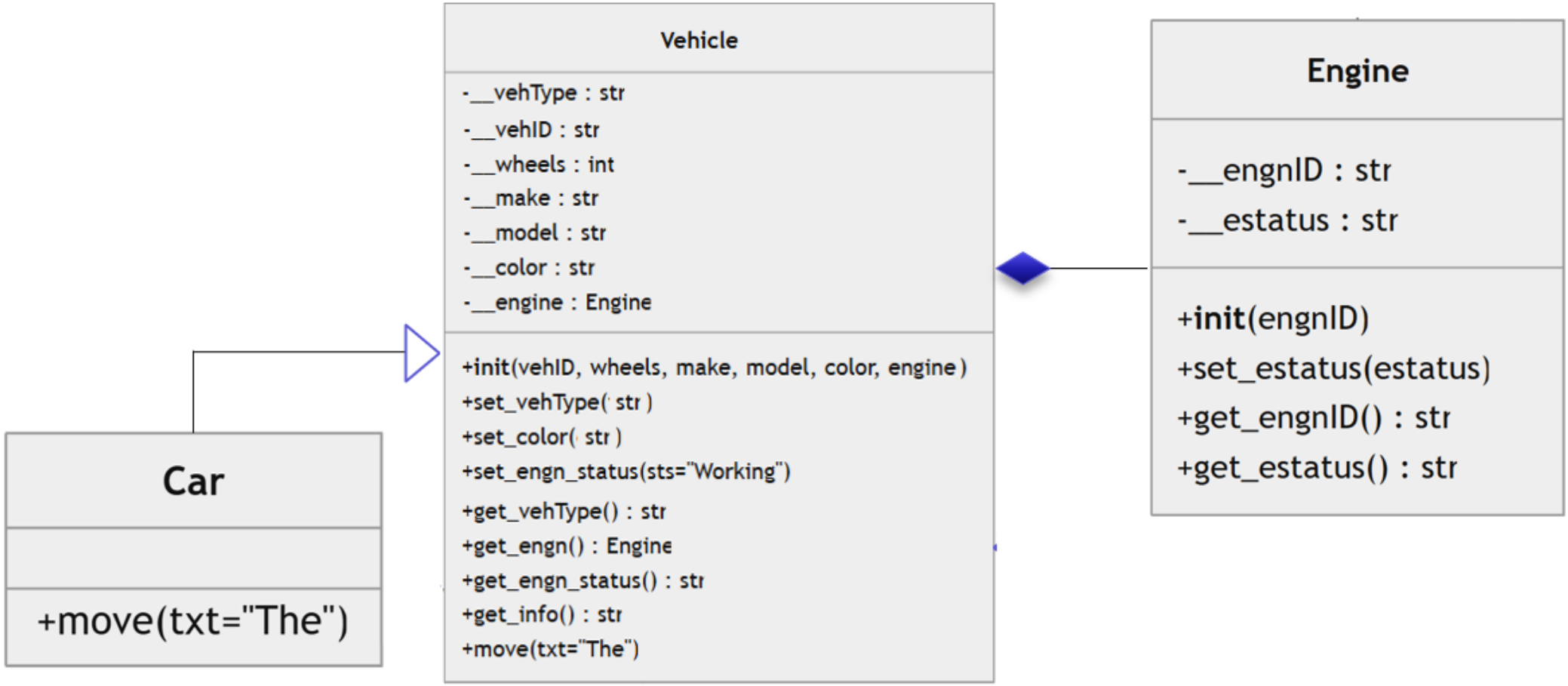
The Implementation:

- The Vehicle class includes an **__engine attribute** in its **__init__** Constructor.
- The move() method performs a **mandatory check** on the **engine's status**.

The Behavior: If the Engine **status** is set to "**Problem**," the Car (a subclass of Vehicle) will **raise an exception**: *"Car could not start: Engine Problem"*.

Key Lesson: A strong relationship implies that the "Part" (Engine) is **essential** for the "Whole" (Vehicle) to perform its identity-defining actions.

Composition



Strong Relationship - Application

Creating Objects

```
engn1 = Engine(1)
my_car = Car(1, 4, "Toyota", "Corolla", "Blue", engn1)
```

```
# Display Info
info = my_car.get_info()
print(info)
```

```
Blue 4-wheeler, Toyota Corolla having Engine:1
```

```
# First Move
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn1.get_estatus())
my_car.move("My " + info)
```

```
Engine 1 Status: Working
My Blue 4-wheeler, Toyota Corolla having Engine:1 Car is being driven.
```

Set to Problem - Simulating Problem

```
my_car.set_engn_status("Problem")
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn1.get_estatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Engine 1 Status: Problem
The Car couldn't start: Engine Problem
```

Set back to Working - Simulating Fix

```
my_car.set_engn_status("Working")
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn1.get_estatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Engine 1 Status: Working
My Blue 4-wheeler, Toyota Corolla having
Engine:1 Car is being driven.
```

Case Study 2: Weak Relationship

Concept: Aggregation represents a "has-a" relationship where the component is an **optional** enhancement. The container can function perfectly well even if the component is missing or broken.

The Scenario: The vehicle is now equipped with an AC (Air Conditioning) unit. Unlike the engine, a car is still a car, and can still drive, even if the AC is broken.

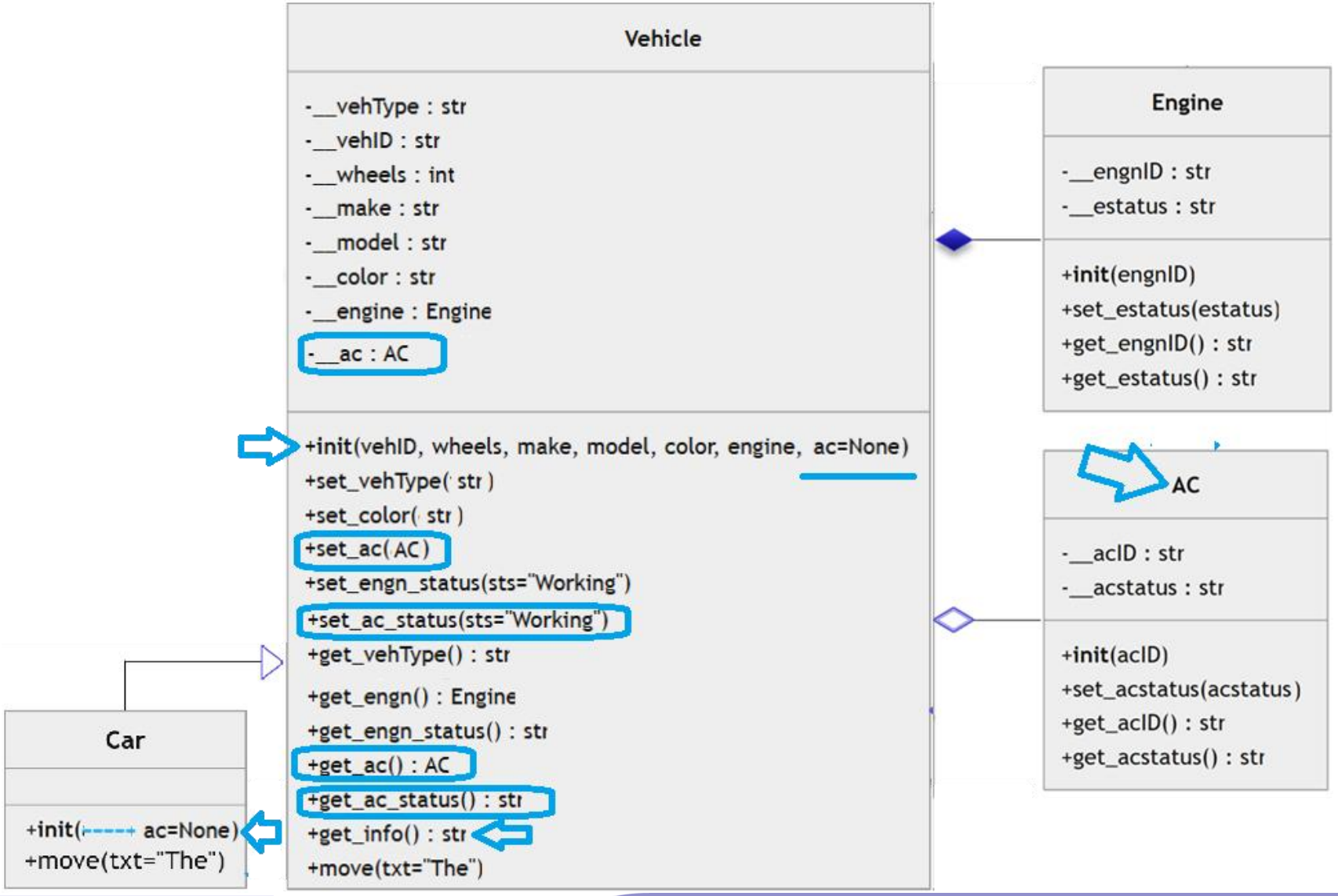
The Implementation:

- The Vehicle class accepts an ac object, but it defaults to None (**ac=None**), signaling it is **optional**.
- The **move()** method does **not check** the status of the AC before allowing the vehicle to drive.

The Behavior: When the AC **status** is set to "**Problem**," the application output shows that the **engine** remains "**Working**." Consequently, when my_car.**move()** is called, it **successfully** prints: *"Car is being driven,"* despite the AC failure.

Key Lesson: In a weak relationship, the lifecycle and status of the component **do not hinder** the primary operations of the base class.

Aggregation



Weak Relationship - Application

Creating Objects

```
engn2 = Engine(2)
ac2 = AC(2)
my_car = Car(2, 4, "Toyota", "Corolla", "Blue", engn2, ac2)
```

```
# Display Info
info = my_car.get_info()
print(info)
```

```
Blue 4-wheeler, Toyota Corolla having
Engine:2, AC:2
```

```
# First Move
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn2.get_estatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac2.get_acstatus())
my_car.move("My " + info)
```

```
Engine 2 Status: Working
AC 2 Status: Working
My Blue 4-wheeler, Toyota Corolla having
Engine:2, AC:2 Car is being driven.
```

Set to Problem - Simulating Problem

```
my_car.set_ac_status("Problem")
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn2.get_estatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac2.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Engine 2 Status: Working
AC 2 Status: Problem
My Blue 4-wheeler, Toyota Corolla having
Engine:2, AC:2 Car is being driven.
```

Case Study 3: Multiple Integral Components

Concept: Systems often rely on **multiple critical parts**. This case study explores how to manage a vehicle that requires both a functional Engine and a functional Battery to operate.

The Scenario: A Battery class is introduced. Like the engine, if the battery is not working, the vehicle should not be able to move.

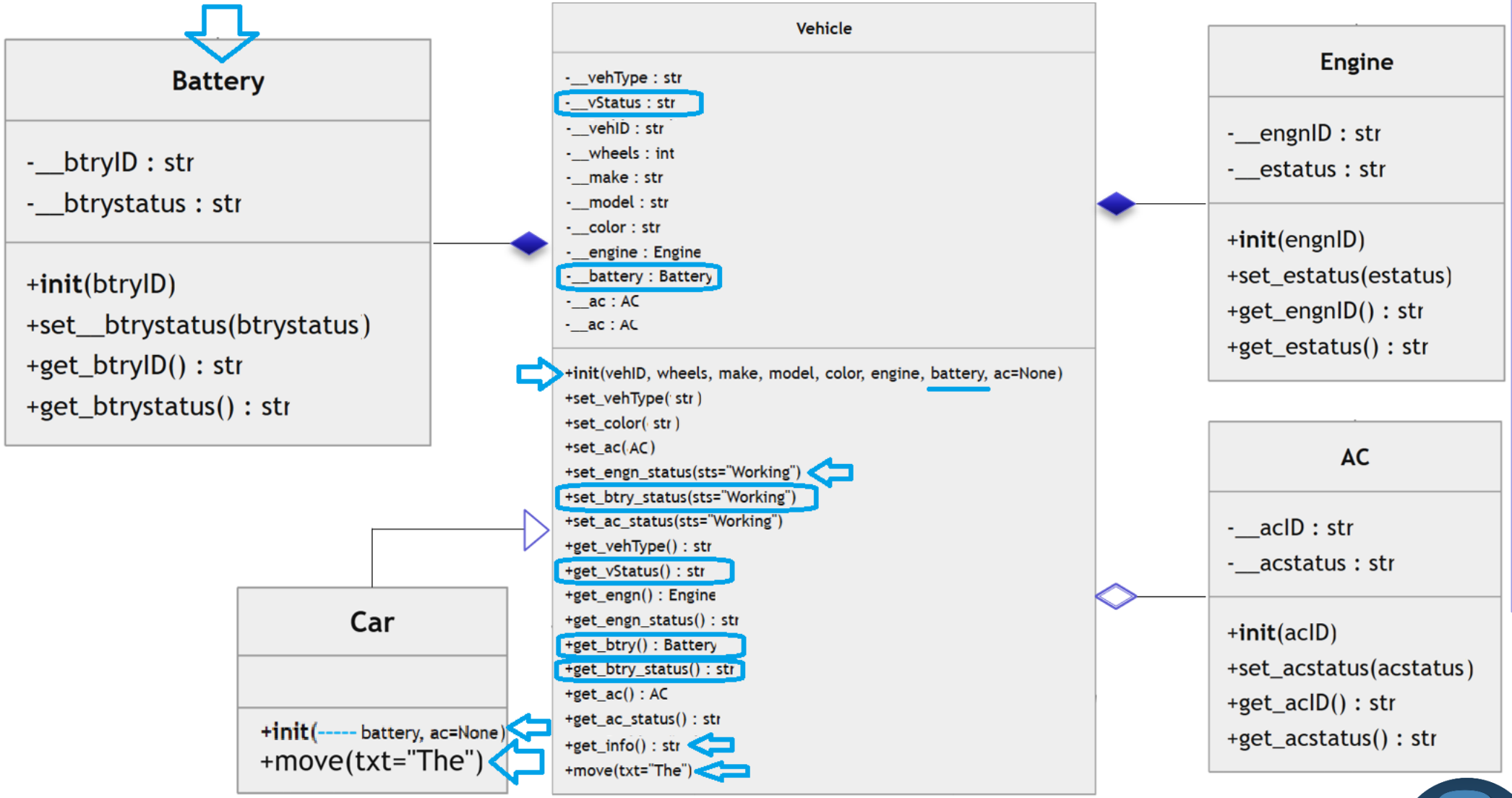
The Implementation:

- A new attribute, **__vStatus** (Vehicle Status), is added to track the **overall health of the vehicle**.
- Setter methods for both Engine and Battery **update** this **__vStatus**. If either component is not "Working," the __vStatus is updated to reflect that specific component's error.

The Behavior: If the Battery is **set** to "**Problem**," the move() method checks get_vStatus(), finds it is not "Working," and **raises an exception**: *"Could not start: Vehicle Problem"*.

Key Lesson: As systems grow, you must aggregate the status of all "**Strong**" **components** into a single state to determine if the "Whole" can function.

Multiple Integral Components



More Integral Components - Application

Creating Objects

```
engn3 = Engine(3)
btry3 = Battery(3)
ac3 = AC(3)
my_car = Car(3, 4, "Toyota", "Corolla", "Blue", engn3, btry3, ac3)

# Display Info
info = my_car.get_info()
print(info)
```

```
Blue 4-wheeler, Toyota Corolla having
Engine:3, Battery:3, AC:3
```

```
# First Move
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn3.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry3.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac3.get_acstatus())
my_car.move("My " + info)
```

```
Car 3 Status: Working
Engine 3 Status: Working
Battery 3 Status: Working
AC 3 Status: Working
My Blue 4-wheeler, Toyota Corolla having
Engine:3, Battery:3, AC:3 Car is being driven.
```

contd...

Set to Problem - Simulating Problem

```
my_car.set_btry_status("Problem")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn3.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry3.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac3.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 3 Status: Problem
Engine 3 Status: Working
Battery 3 Status: Problem
AC 3 Status: Working
The Car couldn't start: Status = Problem
```

Set to Problem - Simulating Problem

```
my_car.set_engn_status("Problem")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn3.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry3.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac3.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 3 Status: Problem
Engine 3 Status: Problem
Battery 3 Status: Working
AC 3 Status: Working
The Car couldn't start: Status = Problem
```

Set back to Working - Simulating Fix

```
my_car.set_btry_status("Working")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn3.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry3.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac3.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 3 Status: Working
Engine 3 Status: Working
Battery 3 Status: Working
AC 3 Status: Working
My Blue 4-wheeler, Toyota Corolla having
Engine:3, Battery:3, AC:3 Car is being driven.
```

Set back to Working - Simulating Fix

```
my_car.set_engn_status("Working")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn3.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry3.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac3.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 3 Status: Working
Engine 3 Status: Working
Battery 3 Status: Working
AC 3 Status: Working
My Blue 4-wheeler, Toyota Corolla having
Engine:3, Battery:3, AC:3 Car is being driven.
```

contd...

Set to Problem - Simulating Problem

```
my_car.set_engn_status("Problem")
my_car.set_btry_status("Problem")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn3.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry3.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac3.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 3 Status: Problem
Engine 3 Status: Problem
Battery 3 Status: Problem
AC 3 Status: Working
The Car couldn't start: Status = Problem
```

Set back to Working - Simulating Fix

```
my_car.set_engn_status("Working")
my_car.set_btry_status("Working")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn3.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry3.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac3.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 3 Status: Working
Engine 3 Status: Working
Battery 3 Status: Working
AC 3 Status: Working
My Blue 4-wheeler, Toyota Corolla having
Engine:3, Battery:3, AC:3 Car is being driven.
```

Case Study 4: Components Status

Concept: This study addresses a logic flaw in Case 3—a single string attribute (**__vStatus**) can only **track** the *last* error that occurred, effectively "hiding" previous failures.

The Problem: If **both** the Engine and Battery **have problems**, the string-based vStatus from Case 3 would only show the most **recently** updated error, making it difficult for a user to know everything that needs fixing.

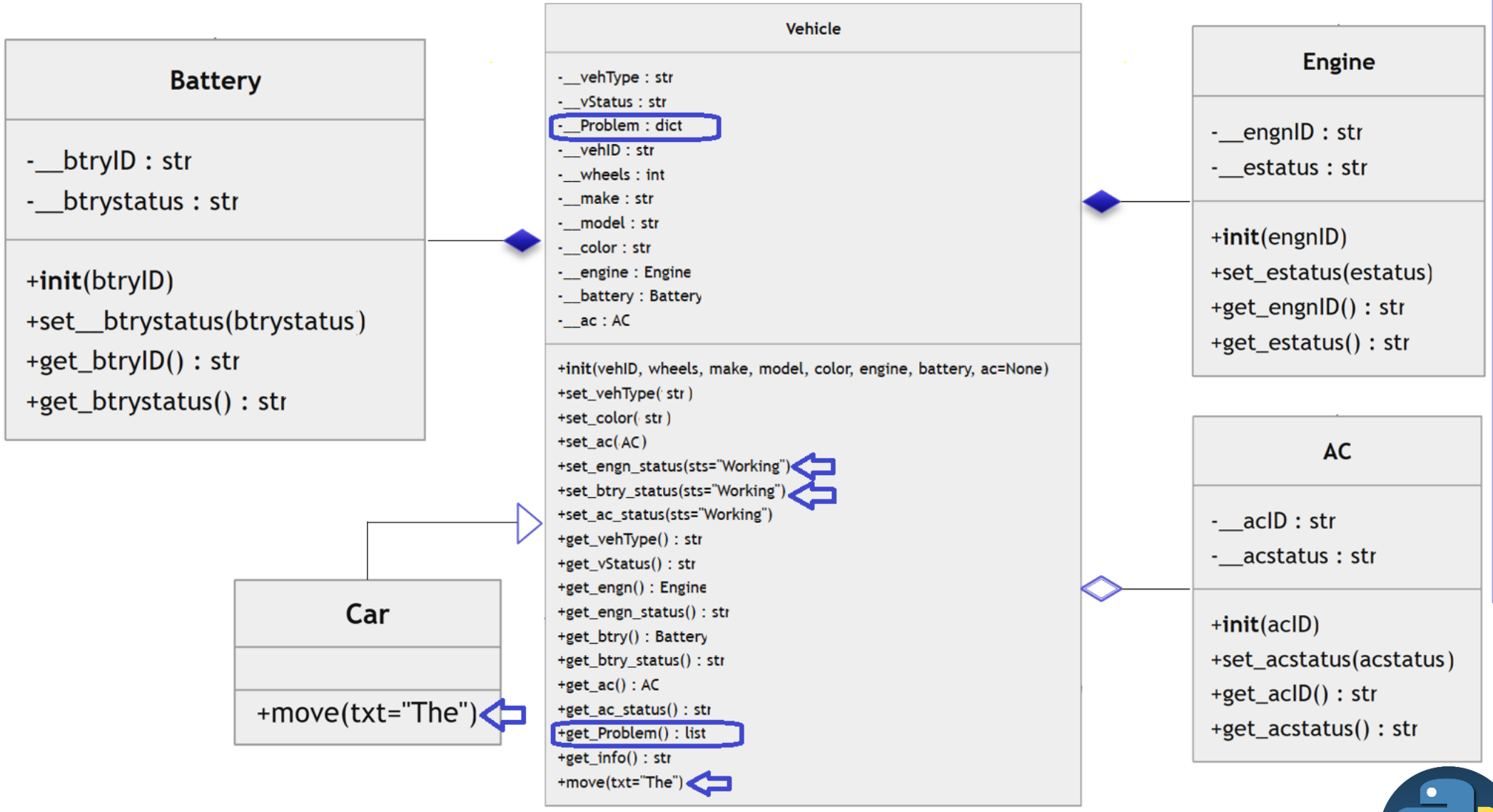
The Solution:

- Replace the simple status string with a **dictionary** called **__Problem**.
- When a component's status is **changed** to a "Problem," it is **added to the dictionary**. When **fixed**, it is **removed**.

The Behavior: In the final simulation, the status of both the Engine and Battery is set to "Problem". When the user attempts to **move** the car, the **exception** now provides a comprehensive **list of all failures**: *"The Car couldn't start: [('Engine', 'Problem'), ('Battery', 'Problem')]"*.

Key Lesson: For complex systems with **multiple integral parts**, use data structures like **dictionaries** to track and display the state of all components simultaneously rather than overwriting a single status variable.

Display Multiple Integral Components Problem



Multiple Integral Components Problem - Application

Creating Objects

```
engn4 = Engine(4)
btry4 = Battery(4)
ac4 = AC(4)
my_car = Car(4, 4, "Toyota", "Corolla", "Blue", engn4, btry4, ac4)

# Display Info
info = my_car.get_info()
print(info)
```

```
Blue 4-wheeler, Toyota Corolla having
Engine:4, Battery:4, AC:4
```

Set to Problem - Simulating Problem

```
my_car.set_engn_status("Problem")
my_car.set_btry_status("Problem")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn4.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry4.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac4.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 4 Status: Problem
Engine 4 Status: Problem
Battery 4 Status: Problem
AC 4 Status: Working
The Car couldn't start: [('Engine', 'Problem'), ('Battery', 'Problem')]
```

```
# First Move
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus()
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn4.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry4.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac4.get_acstatus())
my_car.move("My " + info)
```

```
Car 4 Status: Working
Engine 4 Status: Working
Battery 4 Status: Working
AC 4 Status: Working
My Blue 4-wheeler, Toyota Corolla having
Engine:4, Battery:4, AC:4 Car is being driven.
```

```
my_car.set_engn_status("Working")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus()
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn4.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry4.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac4.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 4 Status: Problem
Engine 4 Status: Working
Battery 4 Status: Problem
AC 4 Status: Working
The Car couldn't start: [('Battery', 'Problem')]
```

contd...

Set back to Working - Simulating Fix

```
my_car.set_btry_status("Working")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn4.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry4.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac4.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 4 Status: Working
Engine 4 Status: Working
Battery 4 Status: Working
AC 4 Status: Working
My Blue 4-wheeler, Toyota Corolla having
Engine:4, Battery:4, AC:4 Car is being driven.
```

Set back to Working - Simulating Fix

```
my_car.set_engn_status("Working")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn4.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry4.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac4.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 4 Status: Working
Engine 4 Status: Working
Battery 4 Status: Working
AC 4 Status: Working
My Blue 4-wheeler, Toyota Corolla having
Engine:4, Battery:4, AC:4 Car is being driven.
```

Set to Problem - Simulating Problem

```
my_car.set_engn_status("Problem")
print(f"{my_car.get_vehType()} {my_car.get_vehID()} Status:", my_car.get_vStatus())
print(f"Engine {my_car.get_engn().get_engnID()} Status:", engn4.get_estatus())
print(f"Battery {my_car.get_btry().get_btryID()} Status:", btry4.get_btrystatus())
print(f"AC {my_car.get_ac().get_acID()} Status:", ac4.get_acstatus())
try: my_car.move("My " + info)
except Exception as e: print(e)
```

```
Car 4 Status: Problem
Engine 4 Status: Problem
Battery 4 Status: Working
AC 4 Status: Working
The Car couldn't start: [('Engine', 'Problem')]
```